Taller OpenMP: Ecuación de Poisson 2D

v1.0

Generated by Doxygen 1.9.8

1 Namespace Index 1
1.1 Namespace List . . . 1

2 File Index 3
2.1 File List . . . 3

3 Namespace Documentation 5
3.1 graficas Namespace Reference . . . 5
3.1.1 Function Documentation . . . 5
3.1.1.1 plot_comparative_3d() . . . 5
3.1.1.2 sol_analitica() . . . 6
3.1.2 Variable Documentation . . . 6
3.1.2.1 archivos . . . 6
3.1.2.2 exist_ok . . . 7
3.1.2.3 salida . . . 7

4 File Documentation 9
4.1 benchmark.sh File Reference . . . 9
4.2 benchmark.sh . . . 9
4.3 graficas.py File Reference . . . 10
4.4 graficas.py . . . 10
4.5 poisson_atomic.cpp File Reference . . . 11
4.5.1 Function Documentation . . . 11
4.5.1.1 main() . . . 11
4.5.1.2 solve_poisson() . . . 12
4.6 poisson_atomic.cpp . . . 13
4.7 poisson_critical.cpp File Reference . . . 13
4.7.1 Function Documentation . . . 14
4.7.1.1 main() . . . 14
4.7.1.2 solve_poisson() . . . 15
4.8 poisson_critical.cpp . . . 15
4.9 poisson_parallel_for.cpp File Reference . . . 16
4.9.1 Function Documentation . . . 16
4.9.1.1 main() . . . 16
4.9.1.2 solve_poisson() . . . 17
4.10 poisson_parallel_for.cpp . . . 18
4.11 poisson_serial.cpp File Reference . . . 18
4.11.1 Function Documentation . . . 19
4.11.1.1 main() . . . 19
4.11.1.2 solve_poisson() . . . 19
4.12 poisson_serial.cpp . . . 20
4.13 poisson_task.cpp File Reference . . . 20
4.13.1 Function Documentation . . . 21

4.13.1.1 main() 21
4.13.1.2 solve_poisson() 22
4.14 poisson_task.cpp 22
4.15 utils.h File Reference 23
4.15.1 Macro Definition Documentation 24
4.15.1.1 VALOR_M 24
4.15.1.2 VALOR_N 24
4.15.2 Function Documentation 25
4.15.2.1 exact_sol() 25
4.15.2.2 export_to_file() 25
4.15.2.3 get_domain() 26
4.15.2.4 get_source_matrix() 27
4.15.2.5 initialize_grid() 27
4.15.2.6 source_func() 28
4.15.3 Variable Documentation 29
4.15.3.1 TOL 29
4.16 utils.h 29

Index **31**

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

graficas 5

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

benchmark.sh 9
graficas.py 10
poisson_atomic.cpp 11
poisson_critical.cpp 13
poisson_parallel_for.cpp 16
poisson_serial.cpp 18
poisson_task.cpp 20
utils.h 23

Chapter 3

Namespace Documentation

3.1 graficas Namespace Reference

Functions

- sol_analitica (ex, x, y)
- plot_comparative_3d (filename, output_name)

Variables

- exist_ok
- archivos = glob.glob('data/∗.dat')
- salida = os.path.join('imag', os.path.basename(arc).replace('.dat', '.png'))

3.1.1 Function Documentation

3.1.1.1 plot_comparative_3d()

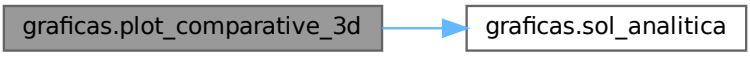
```
graficas.plot_comparative_3d (
            filename,
            output_name )
```

Definition at line 14 of file graficas.py.

```
00014 def plot_comparative_3d(filename, output_name):
00015     try:
00016         data = np.loadtxt(filename)
00017         if data.size == 0: return
00018
00019         # Identificar qué ejemplo es leyendo el nombre del archivo ("_Ej1.dat", etc.)
00020         ex_num = 1
00021         for i in range(1, 5):
00022             if f"Ej{i}" in filename: ex_num = i
00023
00024         x = data[:, 0]; y = data[:, 1]; z = data[:, 2]
00025         M = len(np.unique(x)); N = len(np.unique(y))
00026         X = x.reshape((M, N)); Y = y.reshape((M, N)); Z_num = z.reshape((M, N))
00027
00028         # Generar superficie analítica exacta
00029         Z_ana = sol_analitica(ex_num, X, Y)
00030
00031         fig = plt.figure(figsize=(10, 7))
00032         ax = fig.add_subplot(111, projection='3d')
00033
00034         # Malla negra para la solución analítica
```

```
        ax.plot_wireframe(X, Y, Z_ana, color='black', label='Solución analítica', linewidth=1)

        # Puntos/Superficie roja para la solución numérica
        ax.scatter(X, Y, Z_num, color='red', s=5, label='Solución numérica', alpha=0.8)

        ax.set_xlabel('X')
        ax.set_ylabel('Y')
        ax.set_zlabel('Potencial')
        ax.set_title(f'Ecuación de Poisson - Ejemplo {ex_num}')
        ax.legend()

        plt.savefig(output_name, dpi=300)
        plt.close()
        print(f"Gráfica guardada: {output_name}")
    except Exception as e:
        print(f"Error procesando {filename}: {e}")

```

Here is the call graph for this function:



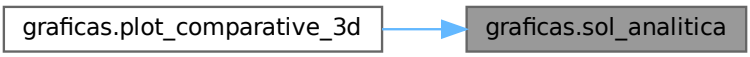
3.1.1.2 sol_analitica()

```
graficas.sol_analitica (
            ex,
            x,
            y )
```

Definition at line 7 of file graficas.py.

```
def sol_analitica(ex, x, y):
    if ex == 1: return np.exp(x * y)
    if ex == 2: return np.log(x**2 + y**2)
    if ex == 3: return (x - y)**2
    if ex == 4: return x * y * np.log(x * y)
    return x*0

```

Here is the caller graph for this function:



3.1.2 Variable Documentation

3.1.2.1 archivos

```
graficas.archivos = glob.glob('data/*.dat')
```

Definition at line 54 of file graficas.py.

3.1.2.2 exist_ok

```
graficas.exist_ok
```

Definition at line 53 of file graficas.py.

3.1.2.3 salida

```
graficas.salida = os.path.join('imag', os.path.basename(arc).replace('.dat', '.png'))
```

Definition at line 56 of file graficas.py.

Chapter 4

File Documentation

4.1 benchmark.sh File Reference

4.2 benchmark.sh

Go to the documentation of this file.

```
#!/bin/bash

METODOS=("poisson_serial" "poisson_parallel_for" "poisson_critical" "poisson_atomic" "poisson_task")
TAMANOS=(50 250 500) # Para el clúster puedes cambiar a: (1000 2000 5000)
HILOS=(12 24 48 64)
LOG="benchmark_cluster_resultados.txt"

echo "Metodo,Hilos,N,Tiempo(s)" > $LOG

echo "Iniciando Benchmark Multihilo..."

# Limpiamos binarios previos
mkdir -p bin data imag
rm -f bin/*

for n in "${TAMANOS[@]}"; do
    echo "========================================="
    echo "  Compilando para tamaño de Malla N=$n"
    echo "========================================="

    # 1. Compilar todo con el nuevo tamaño inyectado por preprocesador
    for m in "${METODOS[@]}"; do
        g++ -std=c++17 -O3 -Wall -Wextra -fopenmp -Isrc -DVALOR_M=$n -DVALOR_N=$n "src/$m.cpp" -o "bin/$m"
    done

    # 2. Ejecutar variando el número de hilos
    for h in "${HILOS[@]}"; do
        export OMP_NUM_THREADS=$h
        echo "--- Probando con $h Hilos ---"

        for m in "${METODOS[@]}"; do
            # El método serial ignora OMP_NUM_THREADS, solo lo corremos 1 vez por tamaño
            if [[ "$m" == "poisson_serial" && "$h" != "${HILOS[0]}" ]]; then
                continue
            fi

            echo -n "  Ejecutando $m ... "

            START=$(date +%s.%N)
            ./bin/$m
            END=$(date +%s.%N)

            DURACION=$(echo "$END - $START" | bc)
            echo "Hecho en $DURACION s"

            # Formato CSV final
            if [[ "$m" == "poisson_serial" ]]; then
                echo "$m,1,$n,$DURACION" » $LOG
            else
```

```
                echo "$m,$h,$n,$DURACION" » $LOG
            fi
        done
    done
done

echo "Benchmark completado. Revisa $LOG"
```

4.3 graficas.py File Reference

Namespaces

- namespace graficas

Functions

- graficas.sol_analitica (ex, x, y)
- graficas.plot_comparative_3d (filename, output_name)

Variables

- graficas.exist_ok
- graficas.archivos = glob.glob('data/∗.dat')
- graficas.salida = os.path.join('imag', os.path.basename(arc).replace('.dat', '.png'))

4.4 graficas.py

Go to the documentation of this file.

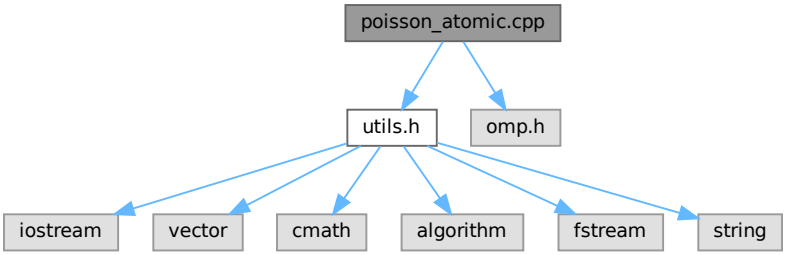
```
import os
import numpy as np
import matplotlib.pyplot as plt
import glob

# Diccionario de soluciones analíticas según el ejemplo
def sol_analitica(ex, x, y):
    if ex == 1: return np.exp(x * y)
    if ex == 2: return np.log(x**2 + y**2)
    if ex == 3: return (x - y)**2
    if ex == 4: return x * y * np.log(x * y)
    return x*0

def plot_comparative_3d(filename, output_name):
    try:
        data = np.loadtxt(filename)
        if data.size == 0: return

        # Identificar qué ejemplo es leyendo el nombre del archivo ("_Ej1.dat", etc.)
        ex_num = 1
        for i in range(1, 5):
            if f"Ej{i}" in filename: ex_num = i

        x = data[:, 0]; y = data[:, 1]; z = data[:, 2]
        M = len(np.unique(x)); N = len(np.unique(y))
        X = x.reshape((M, N)); Y = y.reshape((M, N)); Z_num = z.reshape((M, N))

        # Generar superficie analítica exacta
        Z_ana = sol_analitica(ex_num, X, Y)

        fig = plt.figure(figsize=(10, 7))
        ax = fig.add_subplot(111, projection='3d')

        # Malla negra para la solución analítica
        ax.plot_wireframe(X, Y, Z_ana, color='black', label='Solución analítica', linewidth=1)

        # Puntos/Superficie roja para la solución numérica
```

```
        ax.scatter(X, Y, Z_num, color='red', s=5, label='Solución numérica', alpha=0.8)

        ax.set_xlabel('X')
        ax.set_ylabel('Y')
        ax.set_zlabel('Potencial')
        ax.set_title(f'Ecuación de Poisson - Ejemplo {ex_num}')
        ax.legend()

        plt.savefig(output_name, dpi=300)
        plt.close()
        print(f"Gráfica guardada: {output_name}")
    except Exception as e:
        print(f"Error procesando {filename}: {e}")

if __name__ == '__main__':
    os.makedirs('imag', exist_ok=True)
    archivos = glob.glob('data/*.dat')
    for arc in archivos:
        salida = os.path.join('imag', os.path.basename(arc).replace('.dat', '.png'))
        plot_comparative_3d(arc, salida)
```

4.5 poisson_atomic.cpp File Reference

```
#include "utils.h"
#include <omp.h>
```

Include dependency graph for poisson_atomic.cpp:



Functions

- void solve_poisson (std::vector< std::vector< double > > &T, const std::vector< std::vector< double > > &source, int M, int N, double h, double k, int ex)
- int main ()

4.5.1 Function Documentation

4.5.1.1 main()

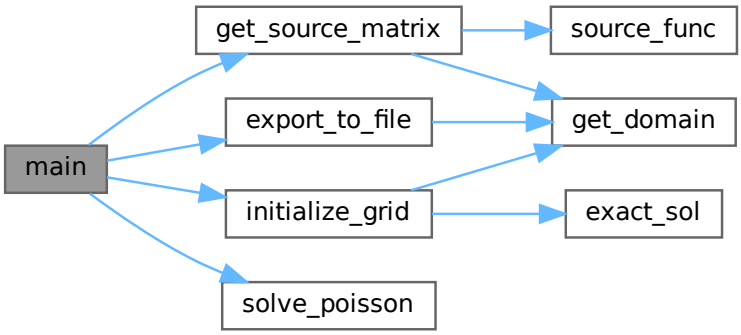
```
int main ( )
```

Definition at line 23 of file poisson_atomic.cpp.

```
{
    int M = VALOR_M, N = VALOR_N;
    for (int ex = 1; ex <= 4; ++ex) {
        std::vector<std::vector<double» T, source; double h, k;
        initialize_grid(ex, M, N, T, h, k); get_source_matrix(ex, M, N, source, h, k);
        solve_poisson(T, source, M, N, h, k, ex);
```

```
        export_to_file(ex, T, h, k, M, N, "atomic");
    }
    return 0;
}
```

Here is the call graph for this function:



4.5.1.2 solve_poisson()

```
void solve_poisson (
            std::vector< std::vector< double > > & T,
            const std::vector< std::vector< double > > & source,
            int M,
            int N,
            double h,
            double k,
            int ex )
```

Definition at line 4 of file poisson_atomic.cpp.

```
{
    double delta = 1.0;
    int iterations = 0;
    while (delta > TOL) {
        delta = 0.0;
        #pragma omp parallel for collapse(2) reduction(max:delta)
        for (int i = 1; i < M; ++i) {
            for (int j = 1; j < N; ++j) {
                double T_new = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] + T[i][j-1]) * h*h) - (source[i][j] * h*h * k*k)) / (2.0 * (h*h + k*k));
                delta = std::max(delta, std::abs(T_new - T[i][j]));
                T[i][j] = T_new;
            }
        }
        #pragma omp atomic
        iterations++;
    }
    if(M <= 1000) std::cout « "Ej " « ex « " Iteraciones (Atomic): " « iterations « std::endl;
}
```

Here is the caller graph for this function:

4.6 poisson_atomic.cpp

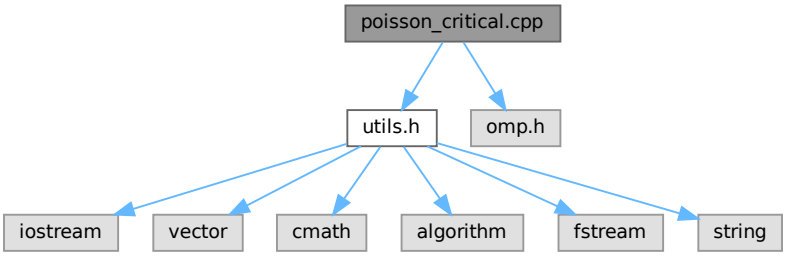
Go to the documentation of this file.

```
#include "utils.h"
#include <omp.h>

void solve_poisson(std::vector<std::vector<double»& T, const std::vector<std::vector<double»& source, int M, int N, double h, double k, int ex) {
    double delta = 1.0;
    int iterations = 0;
    while (delta > TOL) {
        delta = 0.0;
        #pragma omp parallel for collapse(2) reduction(max:delta)
        for (int i = 1; i < M; ++i) {
            for (int j = 1; j < N; ++j) {
                double T_new = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] + T[i][j-1]) * h*h) - (source[i][j] * h*h * k*k)) / (2.0 * (h*h + k*k));
                delta = std::max(delta, std::abs(T_new - T[i][j]));
                T[i][j] = T_new;
            }
        }
        #pragma omp atomic
        iterations++;
    }
    if(M <= 1000) std::cout « "Ej " « ex « " Iteraciones (Atomic): " « iterations « std::endl;
}

int main() {
    int M = VALOR_M, N = VALOR_N;
    for (int ex = 1; ex <= 4; ++ex) {
        std::vector<std::vector<double» T, source; double h, k;
        initialize_grid(ex, M, N, T, h, k); get_source_matrix(ex, M, N, source, h, k);
        solve_poisson(T, source, M, N, h, k, ex);
        export_to_file(ex, T, h, k, M, N, "atomic");
    }
    return 0;
}
```

4.7 poisson_critical.cpp File Reference

```
#include "utils.h"
#include <omp.h>
```

Include dependency graph for poisson_critical.cpp:



Functions

- void solve_poisson (std::vector< std::vector< double > > &T, const std::vector< std::vector< double > > &source, int M, int N, double h, double k, int ex)
- int main ()

4.7.1 Function Documentation

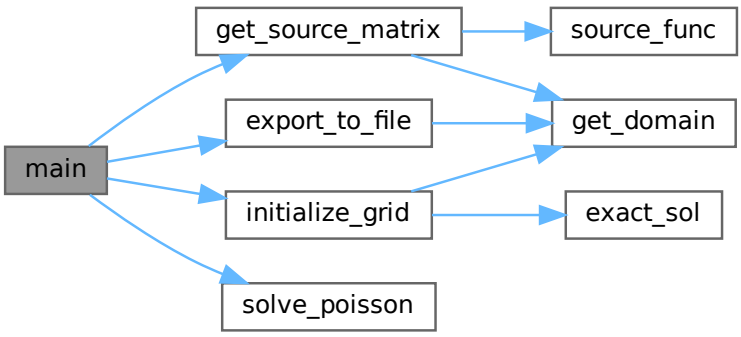
4.7.1.1 main()

```
int main ( )
```

Definition at line 23 of file poisson_critical.cpp.

```
00023           {
00024     int M = VALOR_M, N = VALOR_N;
00025     for (int ex = 1; ex <= 4; ++ex) {
00026         std::vector<std::vector<double» T, source; double h, k;
00027         initialize_grid(ex, M, N, T, h, k); get_source_matrix(ex, M, N, source, h, k);
00028         solve_poisson(T, source, M, N, h, k, ex);
00029         export_to_file(ex, T, h, k, M, N, "critical");
00030     }
00031     return 0;
00032 }
```

Here is the call graph for this function:



4.7.1.2 solve_poisson()

```
void solve_poisson (
            std::vector< std::vector< double > > & T,
            const std::vector< std::vector< double > > & source,
            int M,
            int N,
            double h,
            double k,
            int ex )
```

Definition at line 4 of file poisson_critical.cpp.

```
00004 {
00005     double delta = 1.0;
00006     int iterations = 0;
00007     while (delta > TOL) {
00008         delta = 0.0;
00009         #pragma omp parallel for collapse(2) reduction(max:delta)
00010         for (int i = 1; i < M; ++i) {
00011             for (int j = 1; j < N; ++j) {
00012                 double T_new = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] + T[i][j-1]) * h*h) - (source[i][j] * h*h * k*k)) / (2.0 * (h*h + k*k));
00013                 delta = std::max(delta, std::abs(T_new - T[i][j]));
00014                 T[i][j] = T_new;
00015             }
00016         }
00017         #pragma omp critical
00018         { iterations++; }
00019     }
00020     if(M <= 1000) std::cout « "Ej " « ex « " Iteraciones (Critical): " « iterations « std::endl;
00021 }
```

Here is the caller graph for this function:

4.8 poisson_critical.cpp

Go to the documentation of this file.

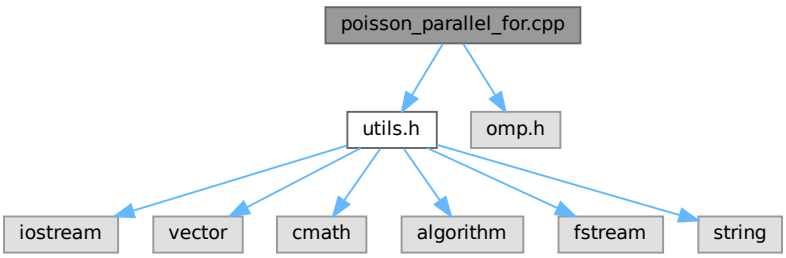
```
00001 #include "utils.h"
00002 #include <omp.h>
00003
00004 void solve_poisson(std::vector<std::vector<double»& T, const std::vector<std::vector<double»& source, int M, int N, double h, double k, int ex) {
00005     double delta = 1.0;
00006     int iterations = 0;
00007     while (delta > TOL) {
00008         delta = 0.0;
00009         #pragma omp parallel for collapse(2) reduction(max:delta)
00010         for (int i = 1; i < M; ++i) {
00011             for (int j = 1; j < N; ++j) {
00012                 double T_new = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] + T[i][j-1]) * h*h) - (source[i][j] * h*h * k*k)) / (2.0 * (h*h + k*k));
00013                 delta = std::max(delta, std::abs(T_new - T[i][j]));
00014                 T[i][j] = T_new;
00015             }
00016         }
00017         #pragma omp critical
00018         { iterations++; }
```

```
    }
    if(M <= 1000) std::cout « "Ej " « ex « " Iteraciones (Critical): " « iterations « std::endl;
}

int main() {
    int M = VALOR_M, N = VALOR_N;
    for (int ex = 1; ex <= 4; ++ex) {
        std::vector<std::vector<double» T, source; double h, k;
        initialize_grid(ex, M, N, T, h, k); get_source_matrix(ex, M, N, source, h, k);
        solve_poisson(T, source, M, N, h, k, ex);
        export_to_file(ex, T, h, k, M, N, "critical");
    }
    return 0;
}
```

4.9 poisson_parallel_for.cpp File Reference

```
#include "utils.h"
#include <omp.h>
```

Include dependency graph for poisson_parallel_for.cpp:



Functions

- void solve_poisson (std::vector< std::vector< double > > &T, const std::vector< std::vector< double > > &source, int M, int N, double h, double k)
- int main ()

4.9.1 Function Documentation

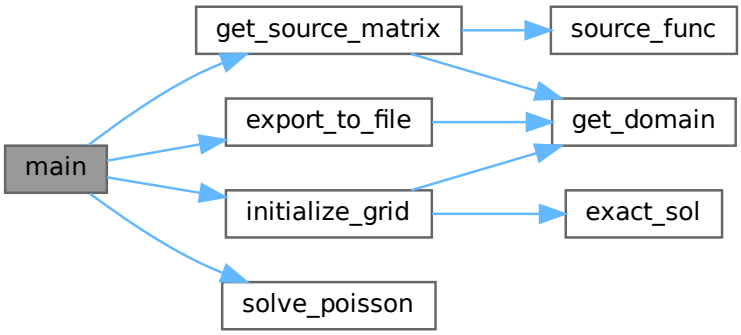
4.9.1.1 main()

```
int main ( )
```

Definition at line 19 of file poisson_parallel_for.cpp.

```
{
    int M = VALOR_M, N = VALOR_N;
    for (int ex = 1; ex <= 4; ++ex) {
        std::vector<std::vector<double» T, source; double h, k;
        initialize_grid(ex, M, N, T, h, k);
        get_source_matrix(ex, M, N, source, h, k);
        solve_poisson(T, source, M, N, h, k);
        export_to_file(ex, T, h, k, M, N, "parallel_for");
    }
    return 0;
}
```

Here is the call graph for this function:



4.9.1.2 solve_poisson()

```
void solve_poisson (
            std::vector< std::vector< double > > & T,
            const std::vector< std::vector< double > > & source,
            int M,
            int N,
            double h,
            double k )
```

Definition at line 4 of file poisson_parallel_for.cpp.

```
00004
      {
00005     double delta = 1.0;
00006     while (delta > TOL) {
00007         delta = 0.0;
00008         #pragma omp parallel for collapse(2) reduction(max:delta)
00009         for (int i = 1; i < M; ++i) {
00010             for (int j = 1; j < N; ++j) {
00011                 double T_new = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] + T[i][j-1]) * h*h) -
      (source[i][j] * h*h * k*k)) / (2.0 * (h*h + k*k));
00012                 delta = std::max(delta, std::abs(T_new - T[i][j]));
00013                 T[i][j] = T_new;
00014             }
00015         }
00016     }
00017 }
```

Here is the caller graph for this function:

4.10 poisson_parallel_for.cpp

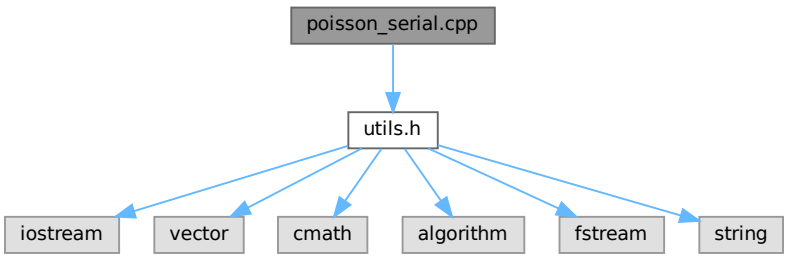
Go to the documentation of this file.

```
#include "utils.h"
#include <omp.h>

void solve_poisson(std::vector<std::vector<double»& T, const std::vector<std::vector<double»& source, int M, int N, double h, double k) {
    double delta = 1.0;
    while (delta > TOL) {
        delta = 0.0;
        #pragma omp parallel for collapse(2) reduction(max:delta)
        for (int i = 1; i < M; ++i) {
            for (int j = 1; j < N; ++j) {
                double T_new = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] + T[i][j-1]) * h*h) - (source[i][j] * h*h * k*k)) / (2.0 * (h*h + k*k));
                delta = std::max(delta, std::abs(T_new - T[i][j]));
                T[i][j] = T_new;
            }
        }
    }
}

int main() {
    int M = VALOR_M, N = VALOR_N;
    for (int ex = 1; ex <= 4; ++ex) {
        std::vector<std::vector<double» T, source; double h, k;
        initialize_grid(ex, M, N, T, h, k);
        get_source_matrix(ex, M, N, source, h, k);
        solve_poisson(T, source, M, N, h, k);
        export_to_file(ex, T, h, k, M, N, "parallel_for");
    }
    return 0;
}
```

4.11 poisson_serial.cpp File Reference

`#include "utils.h"`

Include dependency graph for poisson_serial.cpp:



Functions

- void solve_poisson (std::vector< std::vector< double > > &T, const std::vector< std::vector< double > > &source, int M, int N, double h, double k)
- int main ()

4.11.1 Function Documentation

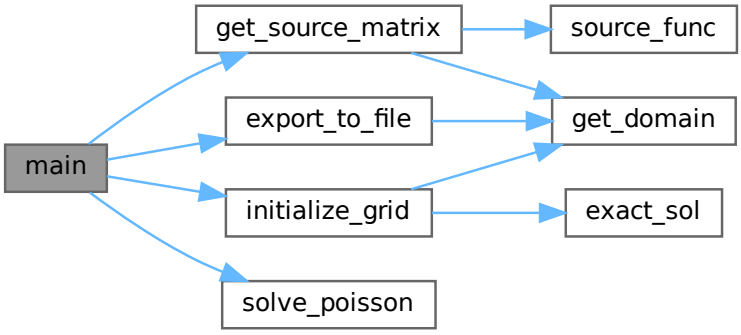
4.11.1.1 main()

```
int main ( )
```

Definition at line 17 of file poisson_serial.cpp.

```
{
    int M = VALOR_M, N = VALOR_N;
    for (int ex = 1; ex <= 4; ++ex) {
        std::vector<std::vector<double» T, source;
        double h, k;
        initialize_grid(ex, M, N, T, h, k);
        get_source_matrix(ex, M, N, source, h, k);
        solve_poisson(T, source, M, N, h, k);
        export_to_file(ex, T, h, k, M, N, "serial");
    }
    return 0;
}
```

Here is the call graph for this function:



4.11.1.2 solve_poisson()

```
void solve_poisson (
            std::vector< std::vector< double > > & T,
            const std::vector< std::vector< double > > & source,
            int M,
            int N,
            double h,
            double k )
```

Definition at line 3 of file poisson_serial.cpp.

```
{
    double delta = 1.0;
    while (delta > TOL) {
        delta = 0.0;
        for (int i = 1; i < M; ++i) {
            for (int j = 1; j < N; ++j) {
                double T_new = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] + T[i][j-1]) * h*h) - (source[i][j] * h*h * k*k)) / (2.0 * (h*h + k*k));
                delta = std::max(delta, std::abs(T_new - T[i][j]));
                T[i][j] = T_new;
```

```
            }
        }
    }
}
```

Here is the caller graph for this function:

4.12 poisson_serial.cpp

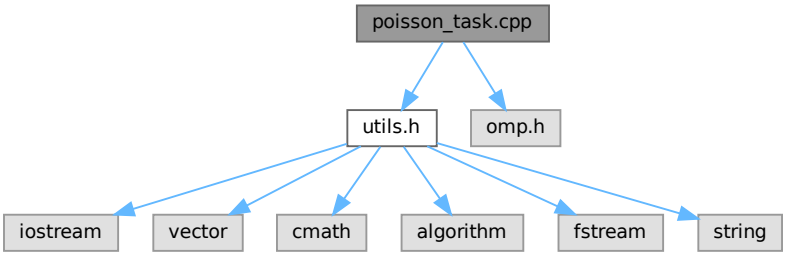
Go to the documentation of this file.

```
#include "utils.h"

void solve_poisson(std::vector<std::vector<double»& T, const std::vector<std::vector<double»& source, int M, int N, double h, double k) {
    double delta = 1.0;
    while (delta > TOL) {
        delta = 0.0;
        for (int i = 1; i < M; ++i) {
            for (int j = 1; j < N; ++j) {
                double T_new = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] + T[i][j-1]) * h*h) - (source[i][j] * h*h * k*k)) / (2.0 * (h*h + k*k));
                delta = std::max(delta, std::abs(T_new - T[i][j]));
                T[i][j] = T_new;
            }
        }
    }
}

int main() {
    int M = VALOR_M, N = VALOR_N;
    for (int ex = 1; ex <= 4; ++ex) {
        std::vector<std::vector<double» T, source;
        double h, k;
        initialize_grid(ex, M, N, T, h, k);
        get_source_matrix(ex, M, N, source, h, k);
        solve_poisson(T, source, M, N, h, k);
        export_to_file(ex, T, h, k, M, N, "serial");
    }
    return 0;
}
```

4.13 poisson_task.cpp File Reference

```
#include "utils.h"
#include <omp.h>
```

Include dependency graph for poisson_task.cpp:



Functions

- void solve_poisson (std::vector< std::vector< double > > &T, const std::vector< std::vector< double > > &source, int M, int N, double h, double k)
- int main ()

4.13.1 Function Documentation

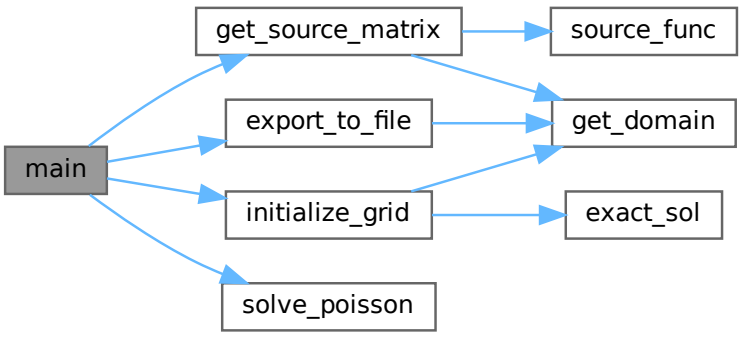
4.13.1.1 main()

```
int main ( )
```

Definition at line 37 of file poisson_task.cpp.

```
00037           {
00038     int M = VALOR_M, N = VALOR_N;
00039     for (int ex = 1; ex <= 4; ++ex) {
00040         std::vector<std::vector<double» T, source; double h, k;
00041         initialize_grid(ex, M, N, T, h, k); get_source_matrix(ex, M, N, source, h, k);
00042         solve_poisson(T, source, M, N, h, k);
00043         export_to_file(ex, T, h, k, M, N, "task");
00044     }
00045     return 0;
00046 }
```

Here is the call graph for this function:



4.13.1.2 solve_poisson()

```
void solve_poisson (
            std::vector< std::vector< double > > & T,
            const std::vector< std::vector< double > > & source,
            int M,
            int N,
            double h,
            double k )
```

Definition at line 4 of file poisson_task.cpp.

```
00004
      {
00005     double delta = 1.0;
00006     int block_size = std::max(10, M / 10); // Escala el bloque dinámicamente
00007     while (delta > TOL) {
00008         double max_delta = 0.0;
00009         #pragma omp parallel shared(max_delta)
00010         {
00011             #pragma omp single
00012             {
00013                 for (int bi = 1; bi < M; bi += block_size) {
00014                     for (int bj = 1; bj < N; bj += block_size) {
00015                         #pragma omp task shared(max_delta)
00016                         {
00017                             double local_delta = 0.0;
00018                             for (int i = bi; i < std::min(bi + block_size, M); ++i) {
00019                                 for (int j = bj; j < std::min(bj + block_size, N); ++j) {
00020                                     double T_new = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] +
      T[i][j-1]) * h*h) - (source[i][j] * h*h * k*k)) / (2.0 * (h*h + k*k));
00021                                     local_delta = std::max(local_delta, std::abs(T_new - T[i][j]));
00022                                     T[i][j] = T_new;
00023                                 }
00024                             }
00025                             #pragma omp critical
00026                             { max_delta = std::max(max_delta, local_delta); }
00027                         }
00028                     }
00029                 }
00030                 #pragma omp taskwait
00031             }
00032         }
00033         delta = max_delta;
00034     }
00035 }
```

Here is the caller graph for this function:

4.14 poisson_task.cpp

Go to the documentation of this file.

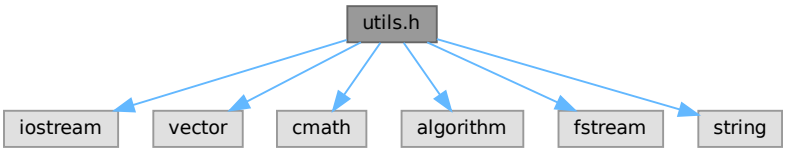
```
00001 #include "utils.h"
00002 #include <omp.h>
00003
00004 void solve_poisson(std::vector<std::vector<double»& T, const std::vector<std::vector<double»& source,
      int M, int N, double h, double k) {
00005     double delta = 1.0;
00006     int block_size = std::max(10, M / 10); // Escala el bloque dinámicamente
00007     while (delta > TOL) {
```

```
        double max_delta = 0.0;
        #pragma omp parallel shared(max_delta)
        {
            #pragma omp single
            {
                for (int bi = 1; bi < M; bi += block_size) {
                    for (int bj = 1; bj < N; bj += block_size) {
                        #pragma omp task shared(max_delta)
                        {
                            double local_delta = 0.0;
                            for (int i = bi; i < std::min(bi + block_size, M); ++i) {
                                for (int j = bj; j < std::min(bj + block_size, N); ++j) {
                                    double T_new = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] + T[i][j-1]) * h*h) - (source[i][j] * h*h * k*k)) / (2.0 * (h*h + k*k));
                                    local_delta = std::max(local_delta, std::abs(T_new - T[i][j]));
                                    T[i][j] = T_new;
                                }
                            }
                            #pragma omp critical
                            { max_delta = std::max(max_delta, local_delta); }
                        }
                    }
                }
                #pragma omp taskwait
            }
        }
        delta = max_delta;
    }
}

int main() {
    int M = VALOR_M, N = VALOR_N;
    for (int ex = 1; ex <= 4; ++ex) {
        std::vector<std::vector<double» T, source; double h, k;
        initialize_grid(ex, M, N, T, h, k); get_source_matrix(ex, M, N, source, h, k);
        solve_poisson(T, source, M, N, h, k);
        export_to_file(ex, T, h, k, M, N, "task");
    }
    return 0;
}
```

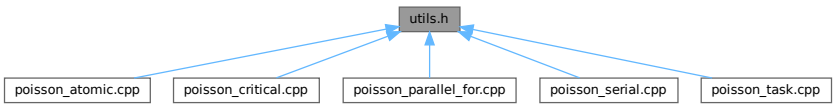
4.15 utils.h File Reference

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <fstream>
#include <string>
```

Include dependency graph for utils.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define VALOR_M 50
- #define VALOR_N 50

Functions

- void get_domain (int ex, double &x_0, double &x_f, double &y_0, double &y_f)
- double exact_sol (int ex, double x, double y)
- double source_func (int ex, double x, double y)
- void initialize_grid (int ex, int M, int N, std::vector< std::vector< double > > &T, double &h, double &k)
- void get_source_matrix (int ex, int M, int N, std::vector< std::vector< double > > &source, double h, double k)
- void export_to_file (int ex, const std::vector< std::vector< double > > &T, double h, double k, int M, int N, const std::string &method_name)

Variables

- const double TOL = 1e-7

4.15.1 Macro Definition Documentation

4.15.1.1 VALOR_M

```
#define VALOR_M 50
```

Definition at line 13 of file utils.h.

4.15.1.2 VALOR_N

```
#define VALOR_N 50
```

Definition at line 16 of file utils.h.

4.15.2 Function Documentation

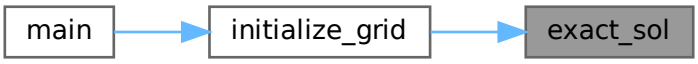
4.15.2.1 exact_sol()

```
double exact_sol (
            int ex,
            double x,
            double y )
```

Definition at line 32 of file utils.h.

```
00032                                                   {
00033     if (ex == 1) return std::exp(x * y);
00034     if (ex == 2) return std::log(x * x + y * y);
00035     if (ex == 3) return (x - y) * (x - y);
00036     if (ex == 4) return x * y * std::log(x * y);
00037     return 0.0;
00038 }
```

Here is the caller graph for this function:



4.15.2.2 export_to_file()

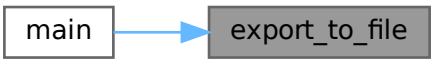
```
void export_to_file (
            int ex,
            const std::vector< std::vector< double > > & T,
            double h,
            double k,
            int M,
            int N,
            const std::string & method_name )
```

Definition at line 79 of file utils.h.

```
00079
      {
00080     // PROTECCIÓN PARA EL CLÚSTER: Si la malla es muy grande (> 1000), no guardamos el .dat
00081     // porque generaría gigabytes de datos y colapsaría el almacenamiento del clúster.
00082     if (M > 1000) return;
00083
00084     double x_0 = 0.0, x_f = 0.0, y_0 = 0.0, y_f = 0.0; // Inicializadas
00085     get_domain(ex, x_0, x_f, y_0, y_f);
00086
00087     std::string filename = "data/solucion_" + method_name + "_Ej" + std::to_string(ex) + ".dat";
00088     std::ofstream file(filename);
00089     if (!file.is_open()) return;
00090
00091     for (int i = 0; i <= M; ++i) {
00092         for (int j = 0; j <= N; ++j) {
00093             file « (x_0 + i * h) « "\t" « (y_0 + j * k) « "\t" « T[i][j] « "\n";
00094         }
00095     }
00096     file.close();
00097 }
```

Here is the call graph for this function:

Here is the caller graph for this function:



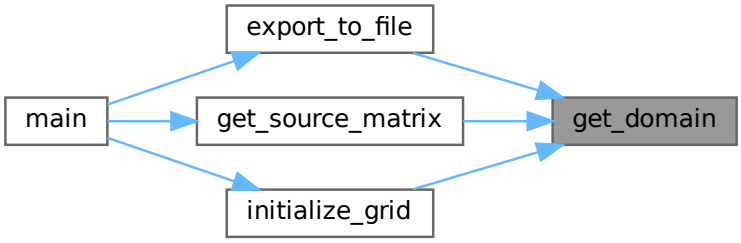
4.15.2.3 get_domain()

```
void get_domain (
            int ex,
            double & x_0,
            double & x_f,
            double & y_0,
            double & y_f )
```

Definition at line 23 of file utils.h.

```
00023                                                                                    {
00024     if (ex == 1) { x_0=0.0; x_f=2.0; y_0=0.0; y_f=1.0; }
00025     else if (ex == 2) { x_0=1.0; x_f=2.0; y_0=0.0; y_f=1.0; }
00026     else if (ex == 3) { x_0=1.0; x_f=2.0; y_0=0.0; y_f=2.0; }
00027     else if (ex == 4) { x_0=1.0; x_f=2.0; y_0=1.0; y_f=2.0; }
00028     else { x_0=0.0; x_f=1.0; y_0=0.0; y_f=1.0; } // Fallback de seguridad
00029 }
```

Here is the caller graph for this function:



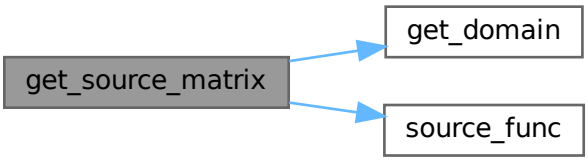
4.15.2.4 get_source_matrix()

```
void get_source_matrix (
            int ex,
            int M,
            int N,
            std::vector< std::vector< double > > & source,
            double h,
            double k )
```

Definition at line 68 of file utils.h.

```
00068
      {
00069     double x_0 = 0.0, x_f = 0.0, y_0 = 0.0, y_f = 0.0; // Inicializadas
00070     get_domain(ex, x_0, x_f, y_0, y_f);
00071     source.assign(M + 1, std::vector<double>(N + 1, 0.0));
00072     for (int i = 0; i <= M; ++i) {
00073         for (int j = 0; j <= N; ++j) {
00074             source[i][j] = source_func(ex, x_0 + i * h, y_0 + j * k);
00075         }
00076     }
00077 }
```

Here is the call graph for this function:



Here is the caller graph for this function:

4.15.2.5 initialize_grid()

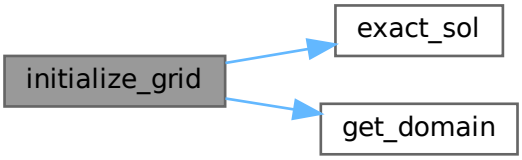
```
void initialize_grid (
            int ex,
            int M,
            int N,
            std::vector< std::vector< double > > & T,
```

```
                double & h,
                double & k )
```

Definition at line 49 of file utils.h.

```
00049                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                {
00050     double x_0 = 0.0, x_f = 0.0, y_0 = 0.0, y_f = 0.0; // Inicializadas
00051     get_domain(ex, x_0, x_f, y_0, y_f);
00052     h = (x_f - x_0) / M;
00053     k = (y_f - y_0) / N;
00054
00055     T.assign(M + 1, std::vector<double>(N + 1, 0.0));
00056
00057     // Condiciones de frontera exactas
00058     for (int j = 0; j <= N; ++j) {
00059         T[0][j] = exact_sol(ex, x_0, y_0 + j * k);
00060         T[M][j] = exact_sol(ex, x_f, y_0 + j * k);
00061     }
00062     for (int i = 0; i <= M; ++i) {
00063         T[i][0] = exact_sol(ex, x_0 + i * h, y_0);
00064         T[i][N] = exact_sol(ex, x_0 + i * h, y_f);
00065     }
00066 }
```

Here is the call graph for this function:



Here is the caller graph for this function:

4.15.2.6 source_func()

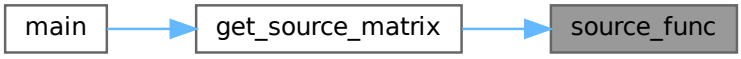
```
double source_func (
                int ex,
                double x,
                double y )
```

Definition at line 41 of file utils.h.

```
00041                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                {
00042     if (ex == 1) return (x * x + y * y) * std::exp(x * y);
00043     if (ex == 2) return 0.0;
```

```
    if (ex == 3) return 4.0;
    if (ex == 4) return (x / y) + (y / x);
    return 0.0;
}
```

Here is the caller graph for this function:



4.15.3 Variable Documentation

4.15.3.1 TOL

```
const double TOL = 1e-7
```

Definition at line 20 of file utils.h.

4.16 utils.h

Go to the documentation of this file.

```
#ifndef UTILS_H
#define UTILS_H

#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <fstream>
#include <string>

// Permite modificar la malla desde la compilación (Bash)
#ifndef VALOR_M
#define VALOR_M 50
#endif
#ifndef VALOR_N
#define VALOR_N 50
#endif

// Tolerancia estricta
const double TOL = 1e-7;

// Obtiene los límites del dominio según el ejemplo
void get_domain(int ex, double& x_0, double& x_f, double& y_0, double& y_f) {
    if (ex == 1) { x_0=0.0; x_f=2.0; y_0=0.0; y_f=1.0; }
    else if (ex == 2) { x_0=1.0; x_f=2.0; y_0=0.0; y_f=1.0; }
    else if (ex == 3) { x_0=1.0; x_f=2.0; y_0=0.0; y_f=2.0; }
    else if (ex == 4) { x_0=1.0; x_f=2.0; y_0=1.0; y_f=2.0; }
    else { x_0=0.0; x_f=1.0; y_0=0.0; y_f=1.0; } // Fallback de seguridad
}

// Soluciones analíticas exactas (Condiciones de frontera)
double exact_sol(int ex, double x, double y) {
    if (ex == 1) return std::exp(x * y);
    if (ex == 2) return std::log(x * x + y * y);
    if (ex == 3) return (x - y) * (x - y);
    if (ex == 4) return x * y * std::log(x * y);
    return 0.0;
}

// Funciones fuente (Lado derecho de la Ecuación de Poisson)
```

```
double source_func(int ex, double x, double y) {
    if (ex == 1) return (x * x + y * y) * std::exp(x * y);
    if (ex == 2) return 0.0;
    if (ex == 3) return 4.0;
    if (ex == 4) return (x / y) + (y / x);
    return 0.0;
}

void initialize_grid(int ex, int M, int N, std::vector<std::vector<double»& T, double& h, double& k) {
    double x_0 = 0.0, x_f = 0.0, y_0 = 0.0, y_f = 0.0; // Inicializadas
    get_domain(ex, x_0, x_f, y_0, y_f);
    h = (x_f - x_0) / M;
    k = (y_f - y_0) / N;

    T.assign(M + 1, std::vector<double>(N + 1, 0.0));

    // Condiciones de frontera exactas
    for (int j = 0; j <= N; ++j) {
        T[0][j] = exact_sol(ex, x_0, y_0 + j * k);
        T[M][j] = exact_sol(ex, x_f, y_0 + j * k);
    }
    for (int i = 0; i <= M; ++i) {
        T[i][0] = exact_sol(ex, x_0 + i * h, y_0);
        T[i][N] = exact_sol(ex, x_0 + i * h, y_f);
    }
}

void get_source_matrix(int ex, int M, int N, std::vector<std::vector<double»& source, double h, double k) {
    double x_0 = 0.0, x_f = 0.0, y_0 = 0.0, y_f = 0.0; // Inicializadas
    get_domain(ex, x_0, x_f, y_0, y_f);
    source.assign(M + 1, std::vector<double>(N + 1, 0.0));
    for (int i = 0; i <= M; ++i) {
        for (int j = 0; j <= N; ++j) {
            source[i][j] = source_func(ex, x_0 + i * h, y_0 + j * k);
        }
    }
}

void export_to_file(int ex, const std::vector<std::vector<double»& T, double h, double k, int M, int N, const std::string& method_name) {
    // PROTECCIÓN PARA EL CLÚSTER: Si la malla es muy grande (> 1000), no guardamos el .dat
    // porque generaría gigabytes de datos y colapsaría el almacenamiento del clúster.
    if (M > 1000) return;

    double x_0 = 0.0, x_f = 0.0, y_0 = 0.0, y_f = 0.0; // Inicializadas
    get_domain(ex, x_0, x_f, y_0, y_f);

    std::string filename = "data/solucion_" + method_name + "_Ej" + std::to_string(ex) + ".dat";
    std::ofstream file(filename);
    if (!file.is_open()) return;

    for (int i = 0; i <= M; ++i) {
        for (int j = 0; j <= N; ++j) {
            file « (x_0 + i * h) « "\t" « (y_0 + j * k) « "\t" « T[i][j] « "\n";
        }
    }
    file.close();
}
#endif
```

Index

archivos
graficas, 6

benchmark.sh, 9

exact_sol
utils.h, 25
exist_ok
graficas, 6
export_to_file
utils.h, 25

get_domain
utils.h, 26
get_source_matrix
utils.h, 26
graficas, 5
archivos, 6
exist_ok, 6
plot_comparative_3d, 5
salida, 7
sol_analitica, 6
graficas.py, 10

initialize_grid
utils.h, 27

main
poisson_atomic.cpp, 11
poisson_critical.cpp, 14
poisson_parallel_for.cpp, 16
poisson_serial.cpp, 19
poisson_task.cpp, 21

plot_comparative_3d
graficas, 5
poisson_atomic.cpp, 11
main, 11
solve_poisson, 12
poisson_critical.cpp, 13
main, 14
solve_poisson, 14
poisson_parallel_for.cpp, 16
main, 16
solve_poisson, 17
poisson_serial.cpp, 18
main, 19
solve_poisson, 19
poisson_task.cpp, 20
main, 21
solve_poisson, 21

salida
graficas, 7
sol_analitica
graficas, 6
solve_poisson
poisson_atomic.cpp, 12
poisson_critical.cpp, 14
poisson_parallel_for.cpp, 17
poisson_serial.cpp, 19
poisson_task.cpp, 21
source_func
utils.h, 28

TOL
utils.h, 29

utils.h, 23
exact_sol, 25
export_to_file, 25
get_domain, 26
get_source_matrix, 26
initialize_grid, 27
source_func, 28
TOL, 29
VALOR_M, 24
VALOR_N, 24

VALOR_M
utils.h, 24
VALOR_N
utils.h, 24